Databricks Architecture

A deep dive into the architectural foundations of the Databricks Lakehouse Platform — from control plane to data plane, cluster internals to cloud-native execution.

AWS · BIG DATA · SPARK

High-Level Architecture Overview

Databricks is built on a clean two-plane separation that divides **management responsibilities** from **compute and storage execution**. This architecture is fundamental to how Databricks delivers security, scalability, and a seamless managed experience across cloud environments.

Control Plane

Hosted and managed by Databricks. Handles orchestration, the web UI, job scheduling, and all platform metadata. Users interact primarily at this layer.

Data Plane

Runs entirely within **your** AWS account. All compute (EC2) and storage (S3) resources reside here — your data never leaves your environment.

Control Plane

The Control Plane is the brain of Databricks — the layer that users and administrators interact with directly. It is fully managed by Databricks and operates outside your cloud account.

Web UI

The Databricks workspace interface — notebooks, dashboards, data explorer, and cluster management all live here.

Job Orchestration

Job scheduler and workflow engine that triggers pipeline runs, manages retries, and coordinates multi-task workflows.

Metadata

Stores cluster configurations, notebook definitions, Unity Catalog metadata, access control policies, and audit logs.

Data Plane

The Data Plane is where all actual computation and data storage occurs. Critically, it runs **inside your own AWS account**, meaning your data is never transmitted to or stored by Databricks infrastructure. This boundary is what makes Databricks enterprise-ready.

Amazon EC2

Databricks provisions EC2 instances dynamically to form Spark clusters. Instance types are configurable — from memory-optimised to GPU-backed — depending on the workload. Clusters auto-scale and terminate after job completion to control costs.

Amazon S3

S3 serves as the persistent storage layer. Delta Lake tables, raw data, intermediate results, and checkpoint files are all stored in S3 buckets within your account. Data is accessed via IAM roles and instance profiles, never via credential sharing.

Why the Separation Matters

The architectural split between the Control Plane and Data Plane is not merely an implementation detail — it is a deliberate design decision with major implications for enterprise cloud deployments.

🔐 Security

Your data never transits through Databricks-managed infrastructure. The Data Plane sits within your VPC, protected by your own IAM policies, security groups, and encryption standards. The Control Plane only sends instructions — never data.

📈 Scalability

Compute resources in the Data Plane scale independently of the Control Plane. Clusters can auto-scale horizontally based on workload demand without any coordination bottleneck, leveraging AWS's elastic infrastructure directly.

⚙️ Managed Services

Databricks handles upgrades, patching, and reliability of the Control Plane, freeing your engineering teams to focus on data pipelines and analytics rather than platform operations.

Execution Flow

Understanding how a job travels from user submission to final output helps clarify how the two planes work in concert. Each step is discrete, traceable, and auditable.

At no point does the Control Plane touch raw data — it only sends execution instructions to the cluster running in your AWS account. The cluster handles all data reads, transformations, and writes autonomously within the Data Plane.

Cluster Internals

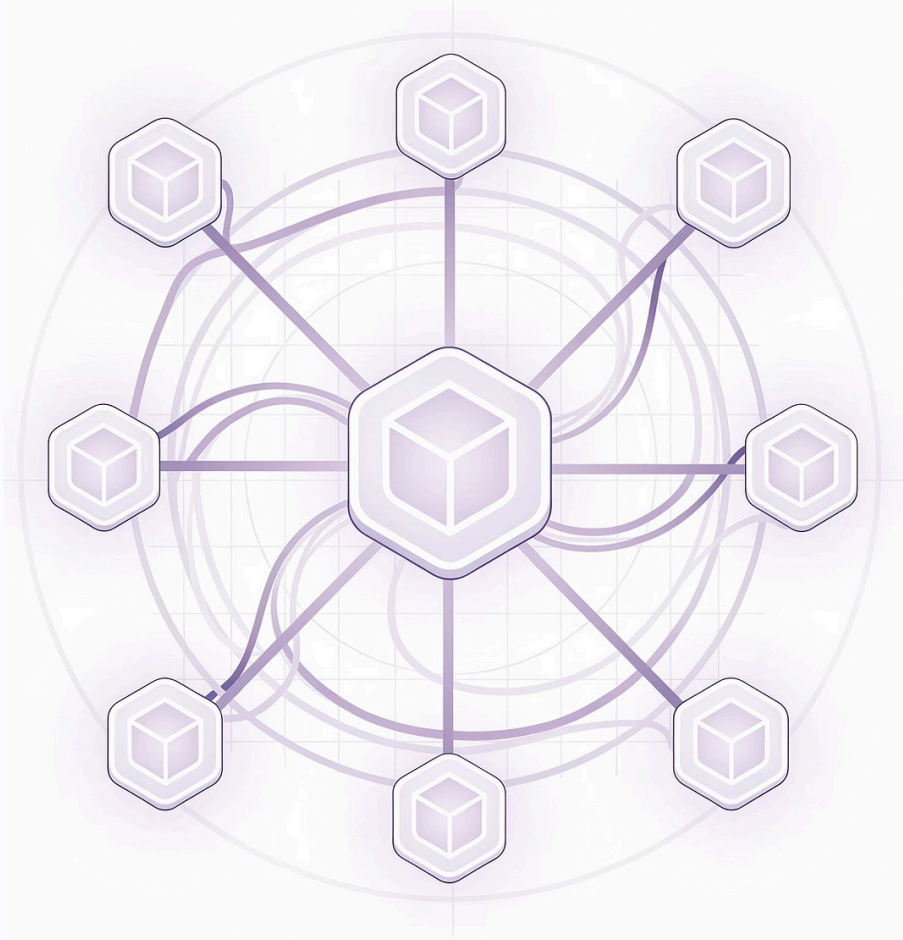
Each Databricks cluster follows the standard Apache Spark architecture, with a clear division of responsibility between the **Driver** and **Worker** nodes running on EC2 instances.

Driver Node

The Driver runs the SparkContext, interprets user code, builds the execution DAG, and coordinates task distribution. It is also responsible for collecting results and returning them to the user. A single driver exists per cluster.

Worker Nodes

Workers execute the actual Spark tasks assigned by the driver. Each worker runs one or more executors, each with a defined allocation of CPU cores and memory. Workers read from and write to S3 in parallel, enabling high-throughput data processing.



Databricks vs. Amazon EMR

Both Databricks and Amazon EMR run Apache Spark on AWS, but they differ significantly in management overhead, developer experience, and platform capabilities.

Feature	Amazon EMR	Databricks
Cluster Management	Manual provisioning; significant ops overhead	Fully managed; auto-scaling and auto-termination
Notebook Experience	Basic; requires third-party tools (e.g. Zeppelin)	Native collaborative notebooks with real-time co-authoring
Delta Lake Support	Supported but not native	Native; built by Databricks, deeply integrated
MLflow Integration	Manual setup required	Built-in; ML experiment tracking out of the box
Optimised Spark Runtime	Open-source Spark only	Databricks Runtime (DBR) with Photon engine
Cost Model	Pay for EC2 only	DBU (Databricks Unit) cost on top of EC2
Governance (Unity Catalog)	Lake Formation integration	Unity Catalog — unified data and AI governance

Key Takeaways

2

Architectural Planes

A clean Control / Data Plane split that keeps your data inside your AWS account at all times.

100%

Managed Spark

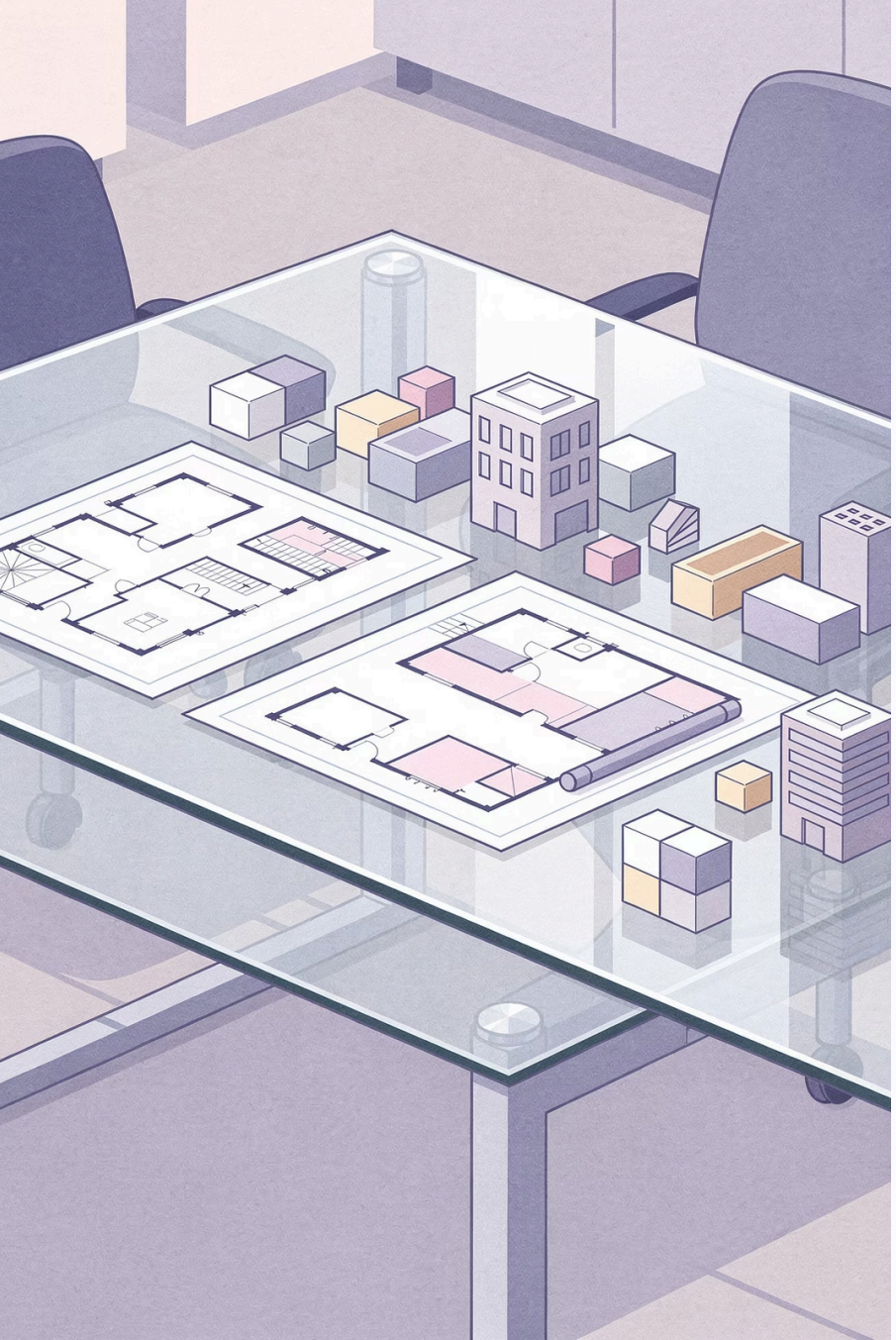
No cluster tuning or infrastructure ops — Databricks Runtime handles Spark optimisation automatically.

AWS

Cloud-Native Design

Built to leverage EC2, S3, IAM, and VPC natively — not a lift-and-shift from on-premises Hadoop.

Databricks is not just a Spark wrapper — it is a full Lakehouse platform combining storage (Delta Lake), compute (DBR + Photon), governance (Unity Catalog), and ML (MLflow) into a single managed experience on AWS.



UP NEXT

Let's Explore the Core Building Blocks

Now that we have established the high-level architecture — the two-plane model, cluster internals, and the comparison with EMR — it is time to go deeper.

01

Delta Lake

ACID transactions, schema enforcement, and time travel on top of S3.

02

Databricks Runtime

The optimised Spark execution engine powering every cluster — including the Photon vectorised engine.

03

Unity Catalog

Centralised governance, lineage tracking, and fine-grained access control across the entire Lakehouse.